ЛР2. Поріг рішення під асиметричну ціну помилок

Дедлайн: до початку лекції 8. **Оцінювання:** 10 балів. **Стек:** Python 3.12, NumPy, pandas, scikit-learn, PyTorch, Matplotlib. Навчання на CPU у `float32`.

Вихідні дані та фіксовані налаштування

Набір Credit Card Fraud Detection (посилання), файл `creditcard.csv`: 284 807 транзакцій, із них 492 шахрайські. Ціль `Class`: 1 означає шахрайство, 0 - легітимну транзакцію. Використайте 29 ознак: `V1`–`V28` і `Amount`. `Class` та `Time` до входу моделі не включайте. Сире значення `Amount` збережіть окремо для аналізу помилок.

Поділ train/validation/test - 60/20/20, стратифікований. Зберігайте початковий порядок рядків CSV. Спочатку виконайте `train_test_split` для індексів із `test_size=0.4`, `stratify=Class`, `random_state=0`. Потім поділіть відкладені індекси навпіл із `test_size=0.5`, стратифікацією за відповідними мітками та `random_state=0`: перша частина - validation, друга - test.

Стандартизуйте ознаки через `StandardScaler`, навчений **лише на train**. До validation і test застосуйте ті самі статистики. Масиви для моделі перетворіть на `float32`. Класи не балансуйте ні відкиданням, ні дублюванням прикладів.

Налаштування	Значення
Модель	`Linear(29,32) → ReLU → Linear(32,1)`
Функція втрат	`BCEWithLogitsLoss`, середнє за пакетом, без зважування класів
Оптимізатор	Adam, швидкість навчання 10^{-3}; решта параметрів за замовчуванням PyTorch
Навчання	12 епох, пакет 1024; останній неповний пакет не відкидати
Відтворюваність	`torch.manual_seed(0)` перед створенням моделі; стандартна ініціалізація `nn.Linear`

Для перемішування train створіть окремий `np.random.default_rng(0)` і отримуйте нову перестановку на кожній епосі. Стартовий код із моделлю та циклом навчання надає викладач. Архітектуру, швидкість навчання, кількість епох і зерна не підбирайте.

---

1. Навчання та збереження прогнозів

Навчіть задану модель один раз на train і використайте ваги після 12-ї епохи. Епоху за validation не вибирайте.

Переведіть модель у `eval()` і без обчислення градієнтів отримайте для validation та test оцінки $s_i = \mathrm{sigmoid}(z_i)$, де z_i - вихідний логіт. Збережіть ваги моделі, індекси поділу, мітки й оцінки без округлення. Аналіз порогів має працювати зі збереженими прогнозами без повторного навчання.

У звіті наведіть кількість об'єктів, кількість шахрайських транзакцій та їхню частку окремо для train, validation і test. Поясніть, чому статистики стандартизації не можна обчислювати на всьому наборі.

2. Вибір порога на валідаційній вибірці

Для порога t прогнозуйте шахрайство, якщо $s_i \geq t$. Самостійно реалізуйте підрахунок TP, FP, FN, TN та вартості помилок:

$$C(t) = 10\,FN(t) + FP(t).$$

FN - пропущене шахрайство, FP - хибна тривога. `Amount` у формулу вартості не входить.

На validation побудуйте криву точність–повнота (precision–recall, PR) і ROC-криву. Для побудови кривих дозволено функції scikit-learn. Наведіть ROC-AUC і поясніть, чому висока ROC-AUC не гарантує малої вартості помилок чи високої точності (precision).

Реалізуйте пошук порога за мінімумом C:

- Кандидати: усі унікальні оцінки validation та $+\infty$. Поріг $+\infty$ означає «усі негативні», найменша оцінка - «усі позитивні».
- Рівні оцінки отримують однаковий прогноз.
- Пошук - через сортування й кумулятивні суми за $O(n \log n)$, де n - розмір validation, без повторного проходу по вибірці для кожного порога.
- Якщо мінімум досягається на кількох порогах, виберіть **найбільший** із них. Позначте результат t^*.

Збережіть повну таблицю кандидатів у CSV: поріг, TP, FP, FN, TN, C. У README - лише підсумкова таблиця; округлюйте тільки для відображення.

Правило на validation	Поріг	TP	FP	FN	TN	C
Стандартний поріг	$0{,}5$					
Мінімум вартості	t^*					
Усі негативні	$+\infty$					

3. Перевірка реалізації пошуку порога

Перевірте реалізацію на трьох малих наборах, незалежно від навченої моделі:

Набір	Оцінки s	Мітки y
A	$[0{,}9;\ 0{,}7;\ 0{,}7;\ 0{,}4;\ 0{,}2;\ 0{,}1]$	$[0;\ 1;\ 0;\ 1;\ 0;\ 0]$
B	$[0{,}9;\ 0{,}6;\ 0{,}3]$	$[0;\ 0;\ 0]$
C*	$[0{,}8;\ \underbrace{0{,}5;\ \ldots\ ;\ 0{,}5}_{11};\ 0{,}2]$	$[1;\ 1;\ \underbrace{0;\ \ldots\ ;\ 0}_{10};\ 0]$

*У наборі C одинадцять оцінок $0{,}5$ підряд: перша з міткою 1, решта десять - з міткою 0.

Для кожного набору побудуйте той самий список кандидатів, що в пункті 2. Окремо, прямим застосуванням правила $s_i \geq t$ для кожного кандидата, обчисліть матрицю плутанини та C.

Повільний перебір дозволений лише тут.

Порівняйте результати з реалізацією через кумулятивні суми: усі лічильники й вартості мають збігатися точно, а вибраний поріг - відповідати тому самому правилу розв'язання нічиєї. Не використовуйте результати швидкого алгоритму для побудови перевірочного.

У звіті наведіть таблиці перевірки, обрані пороги та поясніть, що перевіряють рівні оцінки в наборі A, відсутність позитивних міток у наборі B і два кандидати з однаковою вартістю в наборі C. До тестової оцінки моделі переходьте після проходження всіх трьох перевірок.

---

4. Тестова оцінка та аналіз помилок

Застосуйте t^* до test. Наведіть матрицю плутанини, C, точність і повноту (recall):

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Якщо знаменник нульовий, метрика не визначена - поясніть чому.

Результати test не використовуються для зміни порога, моделі чи налаштувань навчання. Повторний запуск для відтворення дозволений.

Випишіть перші п'ять хибнопозитивних і перші п'ять хибнонегативних прикладів за номером рядка CSV (якщо менше - усі).

Рядок CSV (від 0, без заголовка)	Тип помилки	Мітка	Оцінка s	Amount
	FP / FN			

Порівняйте сирі `Amount` і відстань оцінок від порога для FP та FN. Чи можна за десятьма прикладами судити про причину помилок моделі?

Завершіть звіт відповіддю: що змінив вибір порога в роботі незмінної моделі, якою була ціна цього рішення в термінах FP і FN та чому результат на validation не гарантує такого самого виграшу на test?

Що здати

Теку `lab2/` у власному GitHub-репозиторії всіх лабораторних робіт, яка містить:

- код у `.py`-файлах, файл залежностей із зафіксованими версіями та дві команди через `uv`: навчання з локального CSV і аналіз зі збережених прогнозів без навчання;
- ваги моделі, індекси поділу й збережені прогнози з мітками (кілька МБ, кладіть у репозиторій);
- дві криві на validation, повну CSV-таблицю кандидатів і зафіксований поріг t^*;
- результати трьох перевірок пошуку порога, тестову оцінку та таблицю розібраних помилок;
- `README` (md/pdf/docx) з описом поділу, формулами метрик і вартості, поясненням пошуку та висновками.

Файл `creditcard.csv` до репозиторію не додавайте. Наведіть посилання на набір і спосіб передати шлях до локального CSV у команді запуску. Для завантаження з Kaggle потрібен обліковий запис.

Що саме має відтворюватися

На машині викладача запускаються обидві команди.

- Аналіз зі збережених прогнозів (пункти 2-4) має дати ті самі числа, що у вашому README: поріг t^*, лічильники, C, таблицю помилок.
- Повторне навчання має дати той самий поділ і ті самі статистики стандартизації. (Оцінки s_i після навчання можуть відрізнятися в молодших розрядах: обчислення PyTorch залежать від версії бібліотеки й платформи. Абсолютного збігу не вимагається. Висновки, зроблені за вашими збереженими прогнозами, залишаються чинними.)

Оцінювання та захист

Складова	**Бали**
Правильний поділ, підготовка даних і навчання	2
Криві, точний пошук порога та порівняння на validation	3
Незалежна перевірка пошуку на трьох малих наборах	2
Коректна тестова оцінка, аналіз помилок і висновки	2
Відтворюваність і повнота звіту	1
Разом	**10**

На захисті студент пояснює власні формули, код і результати перевірок. Потрібно вміти пояснити обробку рівних оцінок, граничних порогів і вибір серед кількох мінімумів вартості.

Використання зовнішнього коду та генеративних інструментів дозволене із зазначенням джерел у `README`; воно не скасовує вимоги розуміти поданий розв'язок. Для повного зарахування результат має відтворюватися за наведеною командою на машині викладача.